



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of
Computing

Subject : Database Programming (SECP3623)
Section : Section 1
Task : Project II
Due : 16th November 2025 (10%)

Name	Matric No.
CHAU YING JIA	A23CS0213
ELIJAH SHE YU SHENG	A23CS0073
NG YU HIN	A23CS0148
TAN ZHI MING	A23CS0189

Table of Contents

1.0 Introduction	2
2.0 NoSQL Data Model	4
3.0 CRUD Implementation Summary	7
3.1 Insert Operations	7
3.2 Query Operations	7
3.3 Update Operations	11
3.4 Delete Operations	13
4.0 Advanced MongoDB Operations.....	14
4.1 Indexing	15
4.2 Aggregation Pipelines	15
4.3 Sorting	16
4.4 Query Performance Discussion	20
5.0 Security & Challenges	21
5.1 Security concerns in NoSQL systems	22
5.2 Consistency trade-offs in distributed systems	23
5.3 Database constraints or limitations	26
6.0 Teamwork.....	26
7.0 Conclusion	27
8.0 Presentation URL.....	28

1.0 Introduction

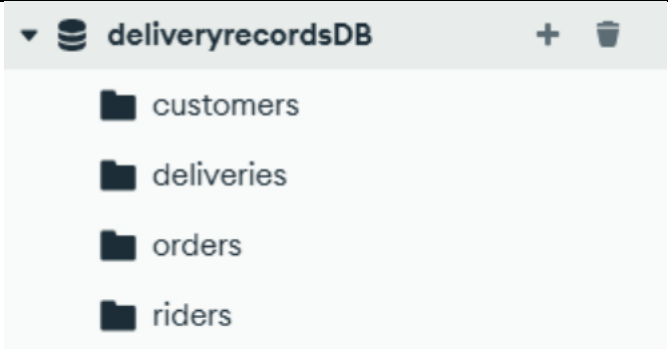
In today's modern society, delivery management has become a critical factor for businesses to maintain customer satisfaction and operational efficiency. Delivery systems involve multiple entities such as customers, orders and riders that generate dynamic and semi structured data. Traditional relational databases often struggle to handle such varying data formats and frequent updates, making it challenging to track orders and manage rider assignments effectively.

This project aims to design a NoSQL database model using MongoDB which focus on storing and managing delivery records efficiently. The system implements four main collections that includes customers, orders, riders, and deliveries. Document data model utilises embedded documents, arrays, references, and appropriate data types. Embedded documents contain extra information within another document while references link orders to customers and deliveries to riders, reflecting real-world relationships between entities.

The primary objective of this project is to demonstrate how a MongoDB document model can provide flexible, scalable, and high-performance storage for a delivery service. The report presents the design, sample documents, and relationships between collections. This can highlight the advantages of using a NoSQL approach for real-time tracking and reporting. By utilising MongoDB, the system ensures quick access to delivery information, easy updates, and the ability to handle complex data without rigid schema constraints, making it suitable for modern delivery applications.

2.0 NoSQL Data Model

Real World Domain Selected: DELIVERY RECORDS

No	Screenshots & Description
1	 “deliveryrecordsDB” is created, along with 4 collections (customers, deliveries,orders,riders) using MongoDB
2	<pre> _id: ObjectId('695def1f10db6666ca9b529e') customerName: "Tan Zhi Ming" email: "tanzhiming@gmail.com" ▼ phoneNumbers: Array (2) 0: "0167983654" 1: "0136252964" ▼ address: Object street: "Jalan Muar" city: "Muar" state: "Johor" postcode: "84060" registeredDate: 2024-11-15T00:00:00.000+00:00 _id: ObjectId('695def6510db6666ca9b529f') customerName: "Chau Ying Jia" email: "chausingjia@gmail.com" ▼ phoneNumbers: Array (2) 0: "0174892749" 1: "0120592759" ▼ address: Object street: "Jalan Universiti" city: "Skudai" state: "Johor" postcode: "84060" registeredDate: 2024-11-15T00:00:00.000+00:00 _id: ObjectId('695deed010db6666ca9b529d') customerName: "Elijah She" email: "elijahshe@gmail.com" ▼ phoneNumbers: Array (2) 0: "0120987652" 1: "0146282914" ▼ address: Object street: "Jalan Permas" city: "Permas Jaya" state: "Johor" postcode: "81110" registeredDate: 2024-11-15T00:00:00.000+00:00 _id: ObjectId('695dee7b10db6666ca9b529c') customerName: "Ng Yu Hin" email: "ngyuhin@gmail.com" ▼ phoneNumbers: Array (2) 0: "0123456789" 1: "0198765432" ▼ address: Object street: "Jalan Universiti" city: "Skudai" state: "Johor" postcode: "81310" registeredDate: 2024-11-15T00:00:00.000+00:00 </pre> Documents of customers collection with appropriate data types (String, Number, Date, Array, Object)

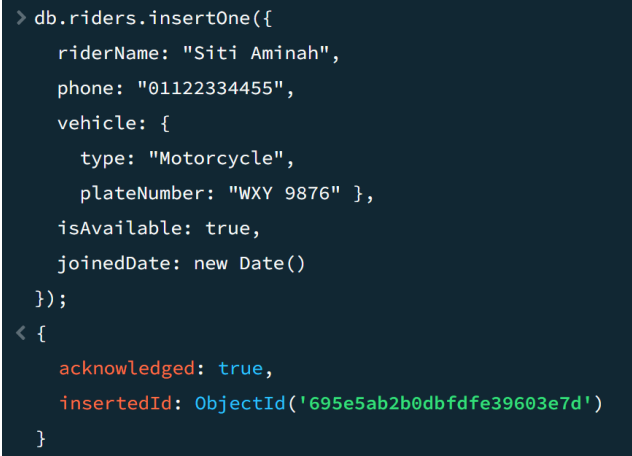
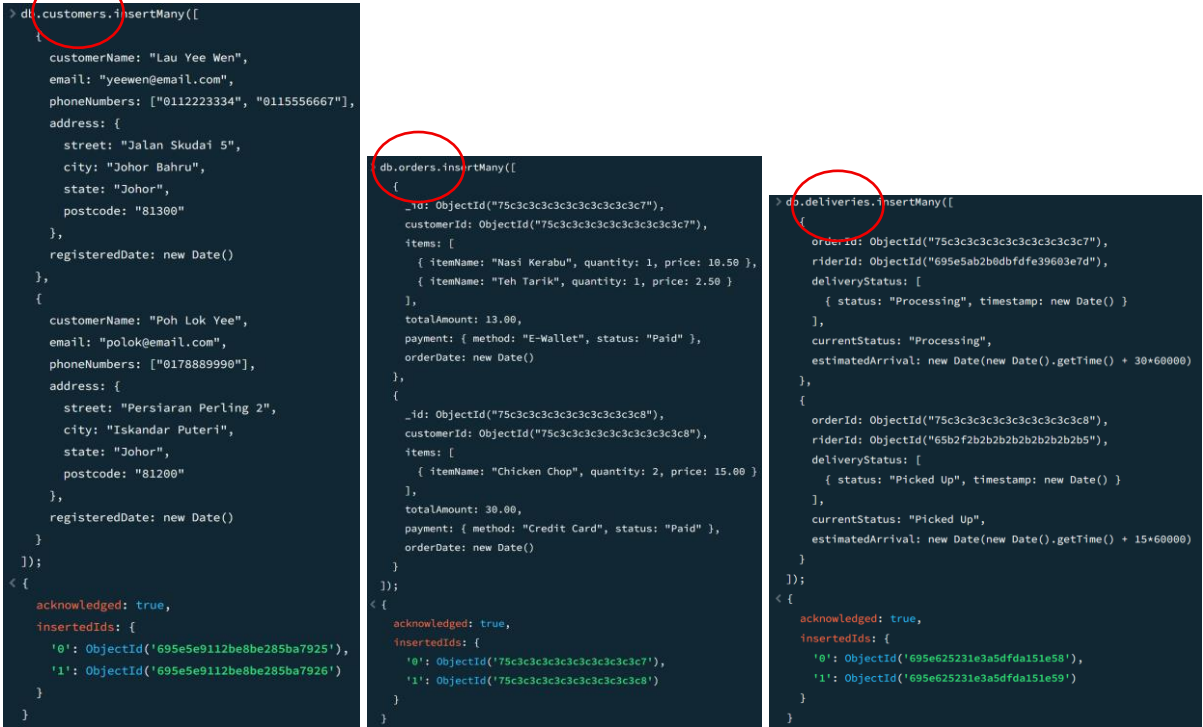
3	<pre> _id: ObjectId('695e22a610db6666ca9b52e6') orderId: ObjectId('65c3c3c3c3c3c3c3c3c3c3c5') riderId: ObjectId('65b2f2b2b2b2b2b2b2b2b2b5') deliveryStatus: Array (3) 0: Object status: "Processing" timestamp: 2025-01-02T11:15:00.000+00:00 1: Object status: "Picked Up" timestamp: 2025-01-02T11:25:00.000+00:00 2: Object status: "Out for Delivery" timestamp: 2025-01-02T11:30:00.000+00:00 currentStatus: "Out for Delivery" estimatedArrival: 2025-01-02T11:50:00.000+00:00 _id: ObjectId('695e27fd10db6666ca9b52e7') orderId: ObjectId('65c3c3c3c3c3c3c3c3c3c3c6') riderId: ObjectId('65b2f2b2b2b2b2b2b2b2b2b6') deliveryStatus: Array (4) 0: Object status: "Processing" timestamp: 2025-01-06T09:15:00.000+00:00 1: Object status: "Picked Up" timestamp: 2025-01-06T09:25:00.000+00:00 2: Object status: "Out for Delivery" timestamp: 2025-01-06T09:30:00.000+00:00 3: Object status: "Delivered" timestamp: 2025-01-06T09:45:00.000+00:00 currentStatus: "Delivered" estimatedArrival: 2025-01-06T09:45:00.000+00:00 </pre> <pre> _id: ObjectId('695df18e10db6666ca9b52ae') orderId: ObjectId('65c3c3c3c3c3c3c3c3c3c3c3') riderId: ObjectId('65b2f2b2b2b2b2b2b2b2b2b2') deliveryStatus: Array (3) 0: Object status: "Processing" timestamp: 2025-01-02T13:09:55.000+00:00 1: Object status: "Picked Up" timestamp: 2025-01-02T13:10:00.000+00:00 2: Object status: "Out for Delivery" timestamp: 2025-01-02T13:30:00.000+00:00 currentStatus: "Out for Delivery" estimatedArrival: 2025-01-02T14:00:00.000+00:00 _id: ObjectId('695df3e910db6666ca9b52af') orderId: ObjectId('65c3c3c3c3c3c3c3c3c3c3c4') riderId: ObjectId('65b2f2b2b2b2b2b2b2b2b2b3') deliveryStatus: Array (4) 0: Object status: "Processing" timestamp: 2025-01-03T13:09:55.000+00:00 1: Object status: "Picked Up" timestamp: 2025-01-03T09:15:00.000+00:00 2: Object status: "Out for Delivery" timestamp: 2025-01-03T09:09:40.000+00:00 3: Object status: "Delivered" timestamp: 2025-01-03T10:05:00.000+00:00 currentStatus: "Delivered" estimatedArrival: 2025-01-03T10:10:05.000+00:00 </pre> Documents of deliveries collection with appropriate data types (String, Number, Date, Array, Object)
4	<pre> _id: ObjectId('75c3c3c3c3c3c3c3c3c3c3c3') customerId: ObjectId('65a1f1a1a1a1a1a1a1a1a1a1') items: Array (2) 0: Object itemName: "Chicken Burger" quantity: 2 price: 8.5 1: Object itemName: "Coke" quantity: 1 price: 2.5 totalAmount: 19.5 payment: Object method: "E-Wallet" status: "Paid" orderDate: 2025-01-02T12:45:00.000+00:00 _id: ObjectId('75c3c3c3c3c3c3c3c3c3c3c4') customerId: ObjectId('65a1f1a1a1a1a1a1a1a1a1a2') items: Array (2) 0: Object itemName: "Beef Burger" quantity: 1 price: 10 1: Object itemName: "Fries" quantity: 1 price: 4 totalAmount: 14 payment: Object method: "Cash" status: "Paid" orderDate: 2025-01-03T09:00:00.000+00:00 </pre> <pre> _id: ObjectId('75c3c3c3c3c3c3c3c3c3c3c5') customerId: ObjectId('65a1f1a1a1a1a1a1a1a1a1a4') items: Array (2) 0: Object itemName: "Spaghetti Carbonara" quantity: 1 price: 11 1: Object itemName: "Garlic Bread" quantity: 1 price: 3.5 totalAmount: 14.5 payment: Object method: "E-Wallet" status: "Paid" orderDate: 2025-01-06T09:00:00.000+00:00 _id: ObjectId('75c3c3c3c3c3c3c3c3c3c3c5') customerId: ObjectId('65a1f1a1a1a1a1a1a1a1a1a3') items: Array (2) 0: Object itemName: "Veggie Pizza" quantity: 1 price: 12.5 1: Object itemName: "Orange Juice" quantity: 2 price: 3 totalAmount: 18.5 payment: Object method: "Credit Card" status: "Paid" orderDate: 2025-01-02T11:00:00.000+00:00 </pre> Documents of orders collection with appropriate data types (String, Number, Date, Array, Object)

5	<pre> _id: ObjectId('65b2f2b2b2b2b2b2b2b2b2b2') riderName: "Muhammad Irfan" phone: "0172233445" vehicle: Object type: "Motorcycle" plateNumber: "JQB 4321" isAvailable: true joinedDate: 2023-06-20T09:30:00.000+00:00 _id: ObjectId('65b2f2b2b2b2b2b2b2b2b2b4') riderName: "Daniel Lee" phone: "0182233441" vehicle: Object type: "Motorcycle" plateNumber: "JQR 9987" isAvailable: false joinedDate: 2024-01-10T08:45:00.000+00:00 </pre>	<pre> _id: ObjectId('65b2f2b2b2b2b2b2b2b2b2b3') riderName: "Aiman Hakim" phone: "0169988776" vehicle: Object type: "Bicycle" plateNumber: "N/A" isAvailable: true joinedDate: 2023-08-15T10:00:00.000+00:00 _id: ObjectId('65b2f2b2b2b2b2b2b2b2b2b5') riderName: "Nur Farah" phone: "0191122334" vehicle: Object type: "Car" plateNumber: "JAB 1234" isAvailable: true joinedDate: 2024-03-05T09:15:00.000+00:00 </pre>
---	--	--

Documents of riders collection
with appropriate data types (String, Number, Date, Array, Object)

3.0 CRUD Implementation Summary

3.1 Insert Operations

No	Screenshots & Description
1	 <pre> > db.riders.insertOne({ riderName: "Siti Aminah", phone: "01122334455", vehicle: { type: "Motorcycle", plateNumber: "WXY 9876" }, isAvailable: true, joinedDate: new Date() }); < { acknowledged: true, insertedId: ObjectId('695e5ab2b0dbdfc39603e7d') } </pre> Insert new document in riders' collection using insertOne() method. Use of embedded documents in the vehicle field.
2	 <pre> > db.customers.insertMany([{ customerName: "Lau Yee Wen", email: "yeewen@email.com", phoneNumbers: ["0112223334", "0115556667"], address: { street: "Jalan Skudai 5", city: "Johor Bahru", state: "Johor", postcode: "81300" }, registeredDate: new Date() }, { customerName: "Poh Lok Yee", email: "polok@email.com", phoneNumbers: ["0178889990"], address: { street: "Persiaran Perling 2", city: "Iskandar Puteri", state: "Johor", postcode: "81200" }, registeredDate: new Date() }]); < { acknowledged: true, insertedIds: { '0': ObjectId('695e5e9112be8be285ba7925'), '1': ObjectId('695e5e9112be8be285ba7926') } } </pre> <pre> db.orders.insertMany([{ _id: ObjectId("75c3c3c3c3c3c3c3c3c7"), customerId: ObjectId("75c3c3c3c3c3c3c3c3c7"), items: [{ itemName: "Nasi Kerabu", quantity: 1, price: 10.50 }, { itemName: "Teh Tarik", quantity: 1, price: 2.50 }], totalAmount: 13.00, payment: { method: "E-Wallet", status: "Paid" }, orderDate: new Date() }, { _id: ObjectId("75c3c3c3c3c3c3c3c3c8"), customerId: ObjectId("75c3c3c3c3c3c3c3c3c8"), items: [{ itemName: "Chicken Chop", quantity: 2, price: 15.00 }], totalAmount: 30.00, payment: { method: "Credit Card", status: "Paid" }, orderDate: new Date() }]); < { acknowledged: true, insertedIds: { '0': ObjectId('75c3c3c3c3c3c3c3c3c7'), '1': ObjectId('75c3c3c3c3c3c3c3c3c8') } } </pre> <pre> > db.deliveries.insertMany([{ orderId: ObjectId("75c3c3c3c3c3c3c3c3c7"), riderId: ObjectId("695e5ab2b0dbdfc39603e7d"), deliveryStatus: [{ status: "Processing", timestamp: new Date() }], currentStatus: "Processing", estimatedArrival: new Date(new Date().getTime() + 30*60000) }, { orderId: ObjectId("75c3c3c3c3c3c3c3c3c8"), riderId: ObjectId("65b2f2b2b2b2b2b2b2b5"), deliveryStatus: [{ status: "Picked Up", timestamp: new Date() }], currentStatus: "Picked Up", estimatedArrival: new Date(new Date().getTime() + 15*60000) }]); < { acknowledged: true, insertedIds: { '0': ObjectId('695e625231e3a5dfda151e58'), '1': ObjectId('695e625231e3a5dfda151e59') } } </pre> Use insertMany() method to add two new customers, orders and deliveries in their respective collections. Usage of embedded address object and array.

3.2 Query Operations

No	Screenshots & Descriptions
1	<pre data-bbox="496 360 1171 808">> db.riders.findOne({ riderName: "Muhammad Irfan" }); < { _id: ObjectId('65b2f2b2b2b2b2b2b2b2b2b2'), riderName: 'Muhammad Irfan', phone: '0172233445', vehicle: { type: 'Motorcycle', plateNumber: 'JQB 4321' }, isAvailable: true, joinedDate: 2023-06-20T09:30:00.000Z }</pre> Use <u>findOne()</u> method to find details for a specific rider by name.
2	<pre data-bbox="576 920 1091 1783">> db.customers.find({ "address.city": { \$eq: "Skudai" } }); < { _id: ObjectId('695dee7b10db6666ca9b529c'), customerName: 'Ng Yu Hin', email: 'ngyuhin@gmail.com', phoneNumbers: ['0123456789', '0198765432'], address: { street: 'Jalan Universiti', city: 'Skudai', state: 'Johor', postcode: '81310' }, registeredDate: 2024-11-15T00:00:00.000Z } { _id: ObjectId('695def6510db6666ca9b529f'), customerName: 'Chau Ying Jia', email: 'chauyingjia@gmail.com', phoneNumbers: ['0174892749', '0120592759'], address: { street: 'Jalan Universiti', city: 'Skudai', state: 'Johor', postcode: '84060' }, registeredDate: 2024-11-15T00:00:00.000Z }</pre> <u>Use find()</u> to retrieve a list of customers living in “Skudai” with the <u>\$eq operator</u> condition. \$eq means equal to.

3

```
> db.orders.find({ totalAmount: { $gt: 15, $lt: 20 } });
< {
  _id: ObjectId('75c3c3c3c3c3c3c3c3c3c3c3'),
  customerId: ObjectId('65a1f1a1a1a1a1a1a1a1a1'),
  items: [
    {
      itemName: 'Chicken Burger',
      quantity: 2,
      price: 8.5
    },
    {
      itemName: 'Coke',
      quantity: 1,
      price: 2.5
    }
  ],
  totalAmount: 19.5,
  payment: {
    method: 'E-Wallet',
    status: 'Paid'
  },
  orderDate: 2025-01-02T12:45:00.000Z
}

{
  _id: ObjectId('75c3c3c3c3c3c3c3c3c3c3c5'),
  customerId: ObjectId('65a1f1a1a1a1a1a1a1a1a3'),
  items: [
    {
      itemName: 'Veggie Pizza',
      quantity: 1,
      price: 12.5
    },
    {
      itemName: 'Orange Juice',
      quantity: 2,
      price: 3
    }
  ],
  totalAmount: 18.5,
  payment: {
    method: 'Credit Card',
    status: 'Paid'
  },
  orderDate: 2025-01-02T11:00:00.000Z
}
```

Filter the data **using \$gt (Greater Than) and \$lt (Less Than)** to find orders within a specific price range which is between 15 and 20.

4

```
> db.customers.find({ customerName: { $in: ["Ng Yu Hin", "Lau Yee Wen"] } });
< {
  _id: ObjectId('695dee7b10db6666ca9b529c'),
  customerName: 'Ng Yu Hin',
  email: 'ngyuhin@gmail.com',
  phoneNumbers: [
    '0123456789',
    '0198765432'
  ],
  address: {
    street: 'Jalan Universiti',
    city: 'Skudai',
    state: 'Johor',
    postcode: '81310'
  },
  registeredDate: 2024-11-15T00:00:00.000Z
}
{
  _id: ObjectId('695e615631e3a5dfda151e56'),
  customerName: 'Lau Yee Wen',
  email: 'yeewen@email.com',
  phoneNumbers: [
    '0112223334',
    '0115556667'
  ],
  address: {
    street: 'Jalan Skudai 5',
    city: 'Johor Bahru',
    state: 'Johor',
    postcode: '81300'
  },
  registeredDate: 2026-01-07T13:36:22.128Z
}
```

Use **\$in** (In an array of values) to search for documents that matches any of the values in the provided list in a single field.

5

```

> db.riders.find({
  $or: [
    { isAvailable: true },
    { "vehicle.type": "Motorcycle" }
  ]
});
< {
  _id: ObjectId('65b2f2b2b2b2b2b2b2b2b2b2'),
  riderName: 'Muhammad Irfan',
  phone: '0172233445',
  vehicle: {
    type: 'Motorcycle',
    plateNumber: 'JQB 4321'
  },
  isAvailable: true,
  joinedDate: 2023-06-20T09:30:00.000Z
}
{
  _id: ObjectId('65b2f2b2b2b2b2b2b2b2b2b3'),
  riderName: 'Aiman Hakim',
  phone: '0169988776',
  vehicle: {
    type: 'Bicycle',
    plateNumber: 'N/A'
  },
  isAvailable: true,
  joinedDate: 2023-08-15T10:00:00.000Z
}
{
  _id: ObjectId('65b2f2b2b2b2b2b2b2b2b2b4'),
  riderName: 'Leonard Lee',
  phone: '0182233441',
  vehicle: {
    type: 'Motorcycle',
    plateNumber: 'JQR 9987'
  },
  isAvailable: false,
  joinedDate: 2024-01-10T08:45:00.000Z
}
{
  _id: ObjectId('65b2f2b2b2b2b2b2b2b2b2b5'),
  riderName: 'Nur Farah',
  phone: '0191122334',
  vehicle: {
    type: 'Car',
    plateNumber: 'JAB 1234'
  },
  isAvailable: true,
  joinedDate: 2024-03-05T09:15:00.000Z
}
{
  _id: ObjectId('69Se5ab2b0dbfdfe39603e7d'),
  riderName: 'Siti Aminah',
  phone: '01122334455',
  vehicle: {
    type: 'Motorcycle',
    plateNumber: 'WXY 9876'
  },
  isAvailable: true,
  joinedDate: 2026-01-07T13:08:02.832Z
}

```

Use **\$OR** to find riders that are available now OR those who drive motorcycle. Needed for a rush delivery.

6

```

> db.orders.find({
  $and: [
    { totalAmount: { $gt: 15 } },
    { "payment.method": "E-Wallet" }
  ]
});
< {
  _id: ObjectId('75c3c3c3c3c3c3c3c3c3c3c3'),
  customerId: ObjectId('65a1f1a1a1a1a1a1a1a1a1a1'),
  items: [
    {
      itemName: 'Chicken Burger',
      quantity: 2,
      price: 8.5
    },
    {
      itemName: 'Coke',
      quantity: 1,
      price: 2.5
    }
  ],
  totalAmount: 19.5,
  payment: {
    method: 'E-Wallet',
    status: 'Paid'
  },
  orderDate: 2025-01-02T12:45:00.000Z
}

```

Use **\$AND** to find orders that are both “totalAmount > 15” AND were paid using an “E-Wallet”.

7

```
> db.orders.find({ "items.itemName": "Teh Tarik" });
< {
  _id: ObjectId('75c3c3c3c3c3c3c3c3c7'),
  customerId: ObjectId('75c3c3c3c3c3c3c3c3c7'),
  items: [
    {
      itemName: 'Nasi Kerabu',
      quantity: 1,
      price: 10.5
    },
    {
      itemName: 'Teh Tarik',
      quantity: 1,
      price: 2.5
    }
  ],
  totalAmount: 13,
  payment: {
    method: 'E-Wallet',
    status: 'Paid'
  },
  orderDate: 2026-01-07T13:38:22.884Z
}
```

Array queries: Find all orders that contain "Teh Tarik" in the array of embdded documents (items) in orders collection.

8

```
> db.customers.find(
  {},
  { customerName: 1, email: 1, _id: 0 }
);
< {
  customerName: 'Ng Yu Hin',
  email: 'ngyuhin@gmail.com'
}
{
  customerName: 'Elijah She',
  email: 'elijahshe@gmail.com'
}
{
  customerName: 'Tan Zhi Ming',
  email: 'tanzhiming@gmail.com'
}
{
  customerName: 'Chau Ying Jia',
  email: 'chauyingjia@gmail.com'
}
{
  customerName: 'Lau Yee Wen',
  email: 'yeewen@email.com'
}
{
  customerName: 'Poh Lok Yee',
  email: 'polok@email.com'
}
```

Projections are to show only the field that is stated, **include or exclude**. The {}, empty clause here is the filter. Find all customers' name and email only. Use 1 to show a field and 0 to hide it.

3.3 Update Operations

No	Screenshots & Description
1	<pre data-bbox="552 376 1115 757"> > db.customers.updateOne({ customerName: "Lau Yee Wen" }, { \$set: { email: "yeewen_new@email.com" } }); < { acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }</pre> _id: ObjectId('695e615631e3a5dfda151e56') customerName : "Lau Yee Wen" email : "yeewen@email.com" ▶ phoneNumbers : Array (2) ▶ address : Object registeredDate : 2026-01-07T13:36:22.128+00:00 => _id: ObjectId('695e615631e3a5dfda151e56') customerName : "Lau Yee Wen" email : "yeewen_new@email.com" ▶ phoneNumbers : Array (2) ▶ address : Object registeredDate : 2026-01-07T13:36:22.128+00:00 Use <u>updateOne()</u> method and \$set operator to change customer's email.
2	<pre data-bbox="584 1104 1083 1585"> > db.orders.updateMany({ totalAmount: { \$gt: 20 } }, { \$inc: { totalAmount: 2 } }); < { acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }</pre> _id: ObjectId('75c3c3c3c3c3c3c3c3c8') customerId : ObjectId('75c3c3c3c3c3c3c3c3c8') items : Array (1) totalAmount : 30 ▶ payment : Object orderDate : 2026-01-07T13:38:22.884+00:00 => _id: ObjectId('75c3c3c3c3c3c3c3c3c8') customerId : ObjectId('75c3c3c3c3c3c3c3c3c8') items : Array (1) totalAmount : 32 ▶ payment : Object orderDate : 2026-01-07T13:38:22.884+00:00 When service tax is needed for higher-value orders, use <u>updateMany()</u> method and <u>\$inc</u> operator to increase all totalAmount that is greater than 20 by 2.

3	<pre> > db.deliveries.updateOne({ _id: ObjectId("695e625231e3a5dfda151e58") }, { \$push: { deliveryStatus: { status: "Out for Delivery", timestamp: new Date() } }, \$set: { currentStatus: "Out for Delivery" } }); < { acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 } </pre> _id: ObjectId('695e625231e3a5dfda151e58') orderId: ObjectId('75c3c3c3c3c3c3c3c3c3c7') riderId: ObjectId('695e5ab2b0dbfdfe39603e7d') ▼ deliveryStatus: Array (1) ▼ 0: Object status: "Processing" timestamp: 2026-01-07T13:40:34.410+00:00 currentStatus: "Processing" estimatedArrival: 2026-01-07T14:10:34.410+00:00 => _id: ObjectId('695e625231e3a5dfda151e58') orderId: ObjectId('75c3c3c3c3c3c3c3c3c3c7') riderId: ObjectId('695e5ab2b0dbfdfe39603e7d') ▼ deliveryStatus: Array (2) ▼ 0: Object status: "Processing" timestamp: 2026-01-07T13:40:34.410+00:00 ▼ 1: Object status: "Out for Delivery" timestamp: 2026-01-07T14:33:03.922+00:00 currentStatus: "Out for Delivery" estimatedArrival: 2026-01-07T14:10:34.410+00:00 Items field is an array, use \$push operator to add a delivery moves to the next stage. Add a new status into the deliveryStatus history array.
4	<pre> > db.customers.updateOne({ customerName: "Elijah She" }, { \$pull: { phoneNumbers: "0146282914" } }); < { acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 } </pre> _id: ObjectId('695deed010db6666ca9b529d') customerName: "Elijah She" email: "elijahshe@gmail.com" ▼ phoneNumbers: Array (2) 0: "0120987652" 1: "0146282914" ▼ address: Object street: "Jalan Permas" city: "Permas Jaya" state: "Johor" postcode: "81110" registeredDate: 2024-11-15T00:00:00.000+00:00 => _id: ObjectId('695deed010db6666ca9b529d') customerName: "Elijah She" email: "elijahshe@gmail.com" ▼ phoneNumbers: Array (1) 0: "0120987652" ▼ address: Object street: "Jalan Permas" city: "Permas Jaya" state: "Johor" postcode: "81110" registeredDate: 2024-11-15T00:00:00.000+00:00 Use \$pull operator to remove customer's registered phone numbers from items list array.

3.4 Delete Operations

No	Screenshots & Descriptions
1	<pre>> db.orders.deleteOne({ _id: ObjectId("75c3c3c3c3c3c3c3c3c6") }); < { acknowledged: true, deletedCount: 1 }</pre> When customer wants to cancel order, use <u>deleteOne()</u> method to cancel the specific (order) document.
2	<pre>> db.riders.deleteMany({ "vehicle.type": { \$ne: "Motorcycle" } }); < { acknowledged: true, deletedCount: 2 }</pre> Use <u>deleteMany()</u> to delete all the riders that are not with motorcycle. \$ne operator means “not equal to”.

4.0 Advanced MongoDB Operations

4.1 Indexing

No	Screenshots & Description
1	<pre>< switched to db FoodDeliveryDB > db.orders.createIndex({ orderDate: 1 }) < orderDate_1 > db.orders.getIndexes() < [{ v: 2, key: { _id: 1 }, name: '_id_' }, { v: 2, key: { orderDate: 1 }, name: 'orderDate_1' }]</pre> <u>Single-field index</u> Creates an ascending index on the <u>orderDate</u> field. This optimizes queries that filter or sort by date, such as generating monthly sales reports or finding the oldest orders.
2	<pre>> db.orders.createIndex({ customerId: 1, totalAmount: -1 }) < customerId_1_totalAmount_-1 > db.orders.getIndexes() < [{ v: 2, key: { _id: 1 }, name: '_id_' }, { v: 2, key: { orderDate: 1 }, name: 'orderDate_1' }, { v: 2, key: { customerId: 1, totalAmount: -1 }, name: 'customerId_1_totalAmount_-1' }]</pre> <u>Compound index</u> Creates a multi-field index that first sorts by <u>customerId</u> (ascending) and then <u>totalAmount</u> (descending). This speeds up complex queries where we need to find a specific customer's most expensive orders instantly.

4.2 Aggregation Pipelines

No	Screenshots & Description
1	<i><u>Pipeline 1: Total Revenue by Payment Method (\$Sort & \$Group)</u></i> <pre data-bbox="276 521 892 898">> db.orders.aggregate([{ \$group: { _id: "\$payment.method", totalRevenue: { \$sum: "\$totalAmount" }, count: { \$sum: 1 } } }, { \$sort: { totalRevenue: -1 } }])</pre> <pre data-bbox="930 465 1370 898">< { _id: 'Credit Card', totalRevenue: 50.5, count: 2 } { _id: 'E-Wallet', totalRevenue: 32.5, count: 2 } { _id: 'Cash', totalRevenue: 14, count: 1 }</pre> Groups all order by payment.method and calculates the total revenue for each type. The result is sorted to show the most popular payment method first.
2	<i><u>Pipeline 2: Delivery Performance Tracking (\$Match, \$Group & \$Project)</u></i> <pre data-bbox="276 1113 847 1408">> db.deliveries.aggregate([{ \$match: { currentStatus: "Delivered" } }, { \$group: { _id: "\$riderId", completedDeliveries: { \$sum: 1 } } }, { \$project: { riderId: "\$_id", completedDeliveries: 1, _id: 0 } }])</pre> <pre data-bbox="863 1162 1374 1408">< { completedDeliveries: 1, riderId: ObjectId('65b2f2b2b2b2b2b2b2b2b6') } { completedDeliveries: 1, riderId: ObjectId('65b2f2b2b2b2b2b2b2b2b3') }</pre> This pipeline filters for deliveries where currentStatus is "Delivered" (\$match). It then groups the results by riderId to count how many successful deliveries each rider has completed. This is useful for calculating rider incentives.

4.3 Sorting

No	Screenshots & Description
1	<i><u>Find the cheapest items bought (Ascending)</u></i> <pre data-bbox="571 448 1091 1608">> db.orders.find().sort({ totalAmount: 1 }) < { _id: ObjectId('75c3c3c3c3c3c3c3c3c7'), customerId: ObjectId('75c3c3c3c3c3c3c3c3c7'), items: [{ itemName: 'Nasi Kerabu', quantity: 1, price: 10.5 }, { itemName: 'Teh Tarik', quantity: 1, price: 2.5 }], totalAmount: 13, payment: { method: 'E-Wallet', status: 'Paid' }, orderDate: 2026-01-07T13:38:22.884Z } { _id: ObjectId('75c3c3c3c3c3c3c3c3c4'), customerId: ObjectId('65a1f1a1a1a1a1a1a1a2'), items: [{ itemName: 'Beef Burger', itemName: 'Orange Juice', quantity: 2, price: 3 }], totalAmount: 18.5, payment: { method: 'Credit Card', status: 'Paid' }, orderDate: 2025-01-02T11:00:00.000Z } { _id: ObjectId('75c3c3c3c3c3c3c3c3c3'), customerId: ObjectId('65a1f1a1a1a1a1a1a1a1'), items: [{ itemName: 'Chicken Burger', quantity: 2, price: 8.5 }, { itemName: 'Coke', quantity: 1, price: 2.5 }], totalAmount: 19.5, payment: {</pre>

```

      itemName: 'Beef Burger',
      quantity: 1,
      price: 10
    },
    {
      itemName: 'Fries',
      quantity: 1,
      price: 4
    }
  ],
  totalAmount: 14,
  payment: {
    method: 'Cash',
    status: 'Paid'
  },
  orderDate: 2025-01-03T09:00:00.000Z
}
{
  _id: ObjectId('75c3c3c3c3c3c3c3c3c5'),
  customerId: ObjectId('65a1f1a1a1a1a1a1a1a3'),
  items: [
    {
      itemName: 'Veggie Pizza',
      quantity: 1,
      price: 12.5
    },
    {
      itemName: 'Orange Juice',
      quantity: 2,
      price: 2.5
    }
  ],
  totalAmount: 19.5,
  payment: {
    method: 'E-Wallet',
    status: 'Paid'
  },
  orderDate: 2025-01-02T12:45:00.000Z
}
{
  _id: ObjectId('75c3c3c3c3c3c3c3c3c8'),
  customerId: ObjectId('75c3c3c3c3c3c3c3c3c8'),
  items: [
    {
      itemName: 'Chicken Chop',
      quantity: 2,
      price: 15
    }
  ],
  totalAmount: 32,
  payment: {
    method: 'Credit Card',
    status: 'Paid'
  },
  orderDate: 2026-01-07T13:38:22.884Z
}

```

Arranges orders from the lowest price to the highest. This allows users to find budget-friendly meal options easily.

2

Find the most recently registered customers (Descending)

```

> db.customers.find().sort({ registeredDate: -1 })
< {
  _id: ObjectId('695e615631e3a5dfda151e56'),
  customerName: 'Lau Yee Wen',
  email: 'yeewen_new@email.com',
  phoneNumbers: [
    '0112223334',
    '0115556667'
  ],
  address: {
    street: 'Jalan Skudai 5',
    city: 'Johor Bahru',
    state: 'Johor',
    postcode: '81300'
  },
  registeredDate: 2026-01-07T13:36:22.128Z
}
{
  _id: ObjectId('695e615631e3a5dfda151e57'),
  customerName: 'Poh Lok Yee',
  email: 'polok@email.com',
  phoneNumbers: [
    '0178889990'
  ],
  address: {
    street: 'Persiaran Perling 2',
    city: 'Iskandar Puteri',
    state: 'Johor',
    postcode: '81200'
  },
  registeredDate: 2026-01-07T13:36:22.128Z
}
{
  _id: ObjectId('695dee7b10db6666ca9b529c'),
  customerName: 'Ng Yu Hin',
  email: 'ngyuhin@gmail.com',
  phoneNumbers: [
    '0123456789',
    '0198765432'
  ],
  address: {
    street: 'Jalan Universiti',
    city: 'Skudai',
    state: 'Johor',
    postcode: '81310'
  },
  registeredDate: 2024-11-15T00:00:00.000Z
}
{
  _id: ObjectId('695deed010db6666ca9b529d'),
  customerName: 'Elifjah She',
  email: 'elifjahshe@gmail.com',
  phoneNumbers: [
    '0120987652'
  ],
  address: {

```

```

{
  _id: ObjectId('695deed010db6666ca9b529d'),
  customerName: 'Elijah She',
  email: 'elijahshe@gmail.com',
  phoneNumbers: [
    '0120987652'
  ],
  address: {
    street: 'Jalan Permas',
    city: 'Permas Jaya',
    state: 'Johor',
    postcode: '81110'
  },
  registeredDate: 2024-11-15T00:00:00.000Z
}
{
  _id: ObjectId('695def1f10db6666ca9b529e'),
  customerName: 'Tan Zhi Ming',
  email: 'tanzhiming@gmail.com',
  phoneNumbers: [
    '0167983654',
    '0136252964'
  ],
  address: {
    street: 'Jalan Muar',
    city: 'Muar',
    state: 'Johor',
    postcode: '84060'
  },
  phoneNumbers: [
    '0167983654',
    '0136252964'
  ],
  address: {
    street: 'Jalan Muar',
    city: 'Muar',
    state: 'Johor',
    postcode: '84060'
  },
  registeredDate: 2024-11-15T00:00:00.000Z
}
{
  _id: ObjectId('695def6510db6666ca9b529f'),
  customerName: 'Chau Ying Jia',
  email: 'chauyingjia@gmail.com',
  phoneNumbers: [
    '0174892749',
    '0120592759'
  ],
  address: {
    street: 'Jalan Universiti',
    city: 'Skudai',
    state: 'Johor',
    postcode: '84060'
  },
  registeredDate: 2024-11-15T00:00:00.000Z
}

```

Arranges customers by their registration date, showing the newest sign-ups first. This is useful for admins to track recent user growth.

4.4 Query Performance Discussion

Scenario: Consider a query where we want to find all orders placed on a specific date, for example: `db.orders.find({ orderDate: "2025-01-02" })`

	Without an Index (Collection Scan)	With the orderDate_1 Index (Index Scan)
1	MongoDB would perform a collection scan. This mean database must visit every single document in the <u>orders</u> collections. From the first to the last to check if the date matches. This is slow & CPU-intensive.	MongoDB would perform Index Scan. The index stores the dates in a sorted order with B-Tree structure in a separate from the actual documents. MongoDB can traverse this sorted list to jump directly to 2025-01-02 and skip nonmatching dates instantly. This reduces the number of documents the server has to read, make query response almost instant.

5.0 Security & Challenges

5.1 Security concerns in NoSQL systems

Compared to conventional Relational Database Management Systems (RDBMS), security in NoSQL databases such as MongoDB is very different. NoSQL provides flexibility, but it also introduces certain vulnerabilities that need to be fixed to safeguard sensitive information like phone numbers and customer addresses in the food delivery system.

1.NoSQL Injection Attacks

NoSQL databases are not vulnerable to SQL injection attacks, where attackers change SQL queries. Instead, they are vulnerable to **BSON Injection or JavaScript Injection**. MongoDB queries are built with BSON objects, so if user input, like a login field, isn't cleaned up properly, an attacker could add bad operators.

Example: An attacker might input `{"$gt": ""}` into a password field. If the application passes this directly to the database, the query becomes "Find a user where the password is greater than an empty string," which is always true, potentially allowing unauthorized access without a valid password.

2.Weak Default Configurations

Historically, MongoDB instances often default to allowing unrestricted access and binding to all IP addresses such as 0.0.0.0. If deployed without changing these defaults, the Food Delivery System's database could be accessed by anyone on the internet, leading to data theft or deletion of the orders and riders' collections.

3.Lack of Schema Enforcement

By default, MongoDB is schema-less. Although this offers flexibility, if the application logic fails to validate data types, it may pose a security risk. In order to cause application crashes or buffer overflow problems in the backend, a malicious user might attempt to insert malformed data such as a large string in place of a valid price.

5.2 Consistency trade-offs in distributed systems

Maintaining data consistency across several servers in a distributed environment such as MongoDB requires inherent trade-offs. The CAP Theorem, which asserts that a distributed system can only concurrently provide two of the following three guarantees consistency, availability, and partition tolerance which best explains these trade-offs.



1. MongoDB and the CAP Theorem

MongoDB is classified as a **Consistency & Partition Tolerance** system.

- **Consistency:** Every read receives the most recent write or an error.
- **Partition Tolerance:** The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network between nodes.

The system is momentarily unavailable for write operations while it holds an election for a new Primary in the event that the Primary node fails or the network splits. The system puts maintaining data consistency ahead of being ready to take new orders during this time.

2. Tuning Consistency vs. Latency

While the default is Consistency & Partition Tolerance system, we can tune the "Food Delivery System" to balance consistency and latency depending on the criticality of the data.

A. Impact on Order Placement

Scenario: When a customer places a new order.

Configuration: w: "majority"

Explanation:

We configure the system to acknowledge the write only after it has been replicated to a majority of the nodes. This ensures strong consistency and data won't be lost if one node crashes, but it increases latency because the application must wait for network acknowledgments.

B. Impact on Browsing Menus

Scenario: A customer is browsing restaurant menus or viewing rider ratings.

Configuration: readPreference: "secondaryPreferred"

Explanation:

The application reads data from secondary nodes rather than the primary. This provides high availability and reduces load on the primary node. However, it introduces eventual consistency which a user might see a menu item that was just updated 1 second ago on the primary but hasn't yet replicated to the secondary node.

3. Implications for the Food Delivery System

For our specific use case, we accepted the following trade-offs:

- **Orders & Payments:** Consistency is our first priority. Payments that are processed but not recorded because of a synchronization lag are unacceptable. In order to ensure data accuracy, we thus accept higher latency during checkout.
- **Rider Location Tracking:** Availability and performance are our top priorities. If a rider's GPS location is off by a few seconds but eventually becomes consistent, the

system will still work. This lets us handle updates that happen very often without overloading the main database node.

5.3 Database constraints or limitations

While MongoDB provides significant flexibility for the Food Delivery System, several technical constraints and limitations were identified during the design and implementation phases.

Constraint / Limitation	Description	Impact on Project	Solution
BSON Document Size Limit	MongoDB enforces a strict 16MB size limit for a single document.	We could not embed the entire history of orders inside a customer document, as frequent users would eventually exceed this cap.	Referencing: We separated orders into their own collection and referenced the customerId. We only used to embed for small, finite lists like orderItems.
No Native Foreign Keys	Unlike SQL, MongoDB does not enforce Referential Integrity such as no cascading deletes by default.	If a Rider account is deleted, their historical deliveries could remain in the database as "orphaned" records pointing to a non-existent user.	Application Logic: We rely on backend code to check for active dependencies before deletion, or we treat historical data as a permanent record even if the user is gone.
Join Performance (\$lookup)	MongoDB lacks efficient SQL-style joins; the \$lookup operation is computationally expensive.	Generating complex reports combining customers, orders, and riders simultaneously causes slower query performance compared to RDBMS.	Denormalization: We duplicated frequently accessed data such as storing the customerAddress inside the orders document to avoid needing a join for every delivery lookup.
Transactional Limits	Multi-document ACID transactions are supported but are resource-intensive and slower than single-document updates.	Coordinating simultaneous updates such as deducting restaurant inventory and creating a payment record is complex.	Atomicity: We designed the schema to rely on Single-Document Atomicity where possible, keeping related data like items and total price within one document.

6.0 Teamwork

Name	Matric No.	Role	Primary Contributions
NG YU HIN	A23CS0148	Member 1	Part A: Authored the Introduction (Section 1.0) and designed the NoSQL Data Model (Section 2.0), including JSON structure definitions and the embedding vs. referencing strategy.
CHAU YING JIA	A23CS0213	Member 2	Part B: Executed and documented all CRUD operations (Section 3.0), providing screenshots for Insert, Query, Update, and Delete actions in MongoDB Compass/Shell.
TAN ZHI MING	A23CS0189	Member 3	Part C (Tech): Implemented Advanced Operations (Section 4.0), including Indexing strategies, Aggregation Pipelines, and Sorting demonstrations.
ELIJAH SHE YU SHENG	A23CS0073	Member 4	Part C (Report): Authored the Security & Limitations (Section 5.0), compiled the Teamwork & Conclusion (Sections 6.0 & 7.0), and performed the final formatting and editing of the report.

7.0 Conclusion

A crucial education in the structural trade-offs and strategic benefits of implementing a NoSQL architecture was given by the development of the Food Delivery System. The team successfully used schema flexibility to enable quick iterations and mastered crucial performance optimizations, like indexing to stop ineffective collection scans, by switching from a strict relational model to MongoDB. Understanding the data modelling, particularly striking a balance between referencing data to reduce duplication and embedding data for read efficiency, was crucial to this success.

But moving away from a SQL mindset was not without its difficulties, especially when it came to complex retrieval and data integrity. The team had to create application-level logic to preserve consistency and avoid orphaned records in the absence of native foreign keys. Managing the stringent 16MB document size limit and building verbose aggregation pipelines for data joins also compelled us to reassess our original design, which ultimately resulted in a more logical and scalable architectural strategy.

Future development must concentrate on horizontal growth and data security to guarantee the system stays stable as it grows. Without sacrificing the advantages of a flexible document model, we advise introducing stringent JSON Schema Validation to ensure data integrity and horizontal sharding to divide the growing workload among servers. Lastly, creating automated backup procedures will be essential for disaster recovery, guaranteeing that the program is robust enough to withstand system failures and real-world demands.

8.0 Presentation URL

<https://youtu.be/hHRB7bEAvjk?si=8DdMmnufWsPuBuu8>